QA 엔지니어를 위한 서버 성능 테스트 입문

서버의 한계를 측정하고, 장애를 예방하는 방법

목차

1. **성능 테스트가 뭔가요?** — 개요
2. **자세히 알아보기** — 상세 이론
3. **어떤 일이 있었나?** — 실제 사례
4. **실습 전에 확인하기** — 환경 점검
5. **성능 테스트 해보기** — Locust
6. **성능 테스트 잘 해보기** — k6

© 2026 Performance Testing Workshop

1. 성능 테스트가 뭔가요?

개요

© 2026 Performance Testing Workshop

성능 테스트란?

서버가 **얼마나 많은 사용자의 요청**을 **안정적으로 처리**할 수 있는지 측정하는 테스트

구분	기능 테스트	성능 테스트
핵심 질문	올바르게 동작하는가?	수백, 수천 명이 동시에 써도 동작하는가?
관심사	정확성	속도, 안정성, 확장성
사용자 수	1명	수십~수만 명

© 2026 Performance Testing Workshop

비유로 이해하기

🛗 엘리베이터

- **기능 테스트**: 버튼을 누르면 해당 층으로 이동하는가?
- **성능 테스트**: 정원 10명인 엘리베이터에 15명이 타면 어떻게 되는가?

> 기능이 "정상"이어도, **부하가 걸리면 다른 결과**가 나올 수 있습니다.

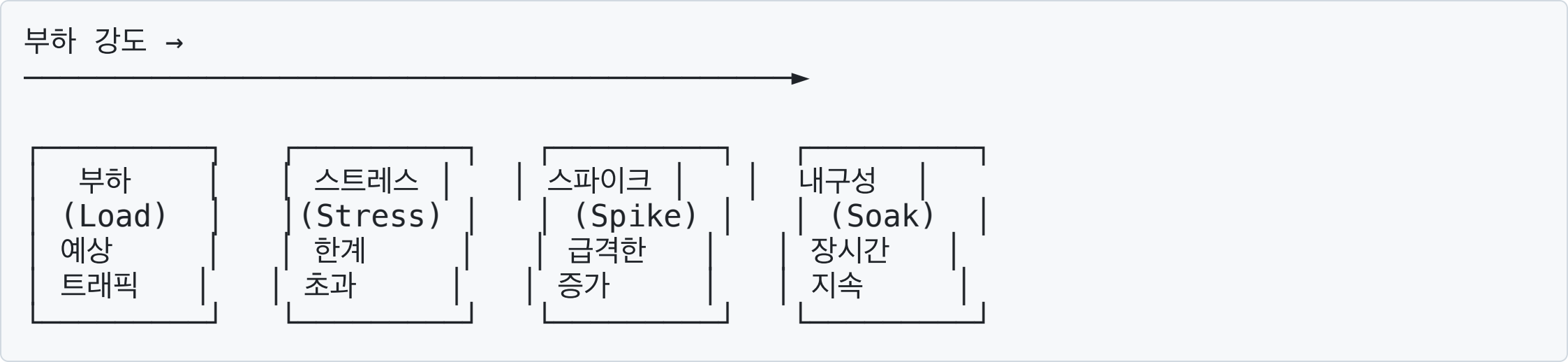
© 2026 Performance Testing Workshop

왜 QA가 성능 테스트를 알아야 할까?

이유	설명
품질의 완성	기능이 정상이어도 느리면 사용자는 떠남
장애 예방	출시 전에 서버의 한계를 파악해야 장애 예방 가능
근거 기반 판단	"느린 것 같아요" → **수치**로 품질을 이야기
커뮤니케이션	개발팀과 성능 이슈를 **정량적**으로 논의

© 2026 Performance Testing Workshop

성능 테스트의 종류



© 2026 Performance Testing Workshop

성능 테스트 종류 — 상세

종류	목적	핵심 질문
부하 테스트 (Load)	예상 트래픽에서 정상 동작 확인	평소 트래픽을 잘 처리하는가?
스트레스 테스트 (Stress)	한계점(Breaking Point) 파악	어디까지 버틸 수 있는가?
스파이크 테스트 (Spike)	급작스러운 트래픽 급증 대응력	갑자기 몰리면 어떻게 되는가?
내구성 테스트 (Soak)	장시간 운영 시 안정성 확인	오래 돌려도 괜찮은가?

© 2026 Performance Testing Workshop

2. 자세히 알아보기

상세 이론

© 2026 Performance Testing Workshop

핵심 성능 지표 — 응답 시간 (Response Time)

서버가 요청을 받고 응답을 돌려주기까지 걸리는 시간

```
클라이언트                          서버
    │──── 요청 (Request) ────►│
    │                          │ ← 처리 중...
    │◄──── 응답 (Response) ────│
    │◄──── 응답 시간 (ms) ────►│
```

- **평균 (Average)**: 전체 응답 시간의 평균
- **중앙값 (Median, p50)**: 50%가 이 시간 이내에 완료
- **p90 / p95 / p99**: 90% / 95% / 99%가 이 시간 이내에 완료

© 2026 Performance Testing Workshop

왜 평균만으로는 부족한가?

요청	응답 시간
1~9번	각 100ms
10번	10,000ms
평균	**1,090ms**
p90	**100ms**
p99	**10,000ms**

> **p99는 "100명 중 가장 불행한 1명의 경험"** 을 보여줍니다.
> 서비스 품질은 **가장 느린 사용자의 경험**이 결정합니다.

© 2026 Performance Testing Workshop

핵심 성능 지표 — 처리량 (Throughput)

단위 시간당 서버가 처리한 요청 수

- **RPS** (Requests Per Second): 초당 처리 요청 수
- **TPS** (Transactions Per Second): 초당 처리 트랜잭션 수

```
RPS = 총 요청 수 / 총 소요 시간(초)
```

> RPS vs TPS
>
> - RPS: HTTP 요청 1건 = 1 Request
> - TPS: 비즈니스 트랜잭션 1건 = 1 Transaction
> (예: 로그인 → 상품조회 → 결제 = 1 Transaction)

© 2026 Performance Testing Workshop

핵심 성능 지표 — 동시 사용자 & 에러율

동시 사용자 수 (Concurrent Users)

같은 시간대에 서버에 요청을 보내고 있는 사용자 수

```
User A: ████ ▒▒▒▒ ████ ▒▒▒▒
User B: ▒▒▒▒ ████ ▒▒▒▒ ████ ▒▒
User C: ▒▒▒▒▒▒ ████ ▒▒▒▒ ████
          ▲ 이 시점의 동시 사용자 = 2명 (A, B)
```

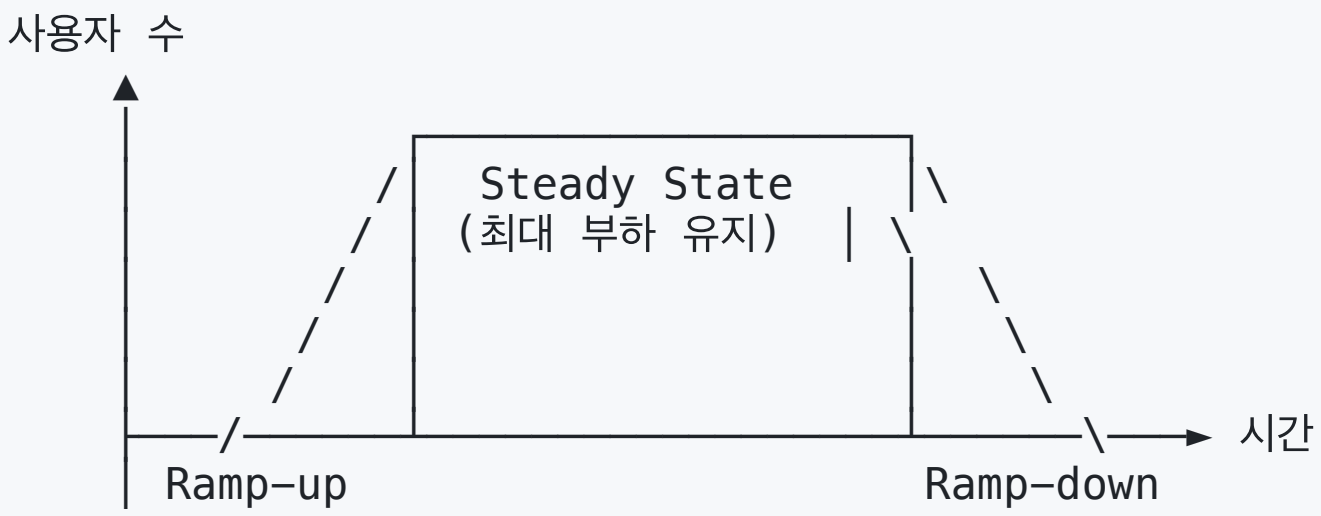
에러율 (Error Rate)

```
에러율(%) = (실패 요청 수 / 전체 요청 수) × 100
```

- 일반적으로 **에러율 1% 이하**를 안정적이라고 판단

© 2026 Performance Testing Workshop

Ramp-up / Steady / Ramp-down



단계	설명	목적
Ramp-up	사용자 점진적 증가	갑작스러운 부하로 인한 왜곡 방지
Steady	목표 사용자 수 유지	안정적인 데이터 수집
Ramp-down	사용자 점진적 감소	부하 해소 후 복구 확인

© 2026 Performance Testing Workshop

Think Time & 가상 사용자 (VU)

Think Time (대기 시간)

실제 사용자가 페이지를 읽거나 입력하는 시간을 시뮬레이션

```
[로그인] ──3초 대기── [상품목록 조회] ──5초 대기── [상품 상세 조회]
              ▲                              ▲
          Think Time                     Think Time
```

- Think Time **없이** 테스트 → 비현실적으로 높은 부하 (잘못된 결과)
- Think Time **포함** 테스트 → 실제 사용자 패턴에 가까운 부하

가상 사용자 (Virtual User, VU)

도구가 생성하는 가상의 사용자. 각 VU는 독립적으로 시나리오를 **반복 수행**

© 2026 Performance Testing Workshop

성능 테스트 프로세스

```
┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│ 1. 목표      │   │ 2. 시나리오  │   │ 3. 환경      │   │ 4. 실행      │   │ 5. 분석      │
│    설정      │ → │    설계      │ → │    구성      │ → │ & 모니터링   │ → │ & 보고       │
│              │   │              │   │              │   │              │   │              │
│ - 대상 API   │   │ - 유저 흐름  │   │ - 테스트     │   │ - 테스트     │   │ - 결과       │
│ - 목표 RPS   │   │ - 데이터     │   │   서버       │   │   실행       │   │   해석       │
│ - 성공기준   │   │ - Think      │   │ - 모니터링   │   │ - 실시간     │   │ - 병목       │
│              │   │   Time       │   │   도구       │   │   관찰       │   │   식별       │
└──────────────┘   └──────────────┘   └──────────────┘   └──────────────┘   └──────────────┘
```

흔한 실수

실수	왜 문제인가	올바른 접근
Think Time 없이 테스트	비현실적 결과	실제 사용자 패턴 반영
단일 API만 테스트	실제 패턴과 다름	시나리오 기반 여러 API 조합
로컬 환경에서 테스트	의미 없는 결과	운영 유사 환경에서 실행
Ramp-up 없이 바로 최대 부하	왜곡된 결과	점진적 부하 증가
1회만 실행 후 결론	신뢰도 낮음	**최소 3회** 반복 후 평균

© 2026 Performance Testing Workshop

3. 어떤 일이 있었나?

실제 사례

© 2026 Performance Testing Workshop

사례 1: 티켓 예매 사이트

"새벽에 열었는데 서버가 터졌어요"

상황: 인기 콘서트 티켓 오픈 시간에 수십만 명 동시 접속

증상

- 페이지 로딩 **30초 이상**
- "504 Gateway Timeout" 대량 발생
- 결제 도중 세션 만료, **중복 결제** 발생

원인: 순간 트래픽 급증 시나리오 미검증 (스파이크 테스트 미실시)

> **교훈**: 이벤트성 서비스는 반드시 **스파이크 테스트**를 수행하고,
> 예상 최대 동시 접속자의 **2~3배** 수준으로 테스트해야 합니다.

© 2026 Performance Testing Workshop

사례 2: 쇼핑몰

"블랙프라이데이에 결제가 안 돼요"

상황: 대형 할인 행사 기간, 평소 대비 **10배** 트래픽

증상

- 상품 조회 응답 시간: 200ms (정상)
- 결제 API 응답 시간: **15,000ms → Timeout**
- 에러율 **45%**까지 상승

원인: "결제는 PG사 영역"이라 판단, 테스트 범위에서 **결제 API를 제외**

> **교훈**: 성능 테스트는 **전체 사용자 흐름(End-to-End)** 을 대상으로 해야 합니다.
> 특정 API를 제외하면 **가장 약한 고리**를 놓치게 됩니다.

© 2026 Performance Testing Workshop

사례 3: 사내 시스템

"월요일 아침마다 느려요"

상황: 월요일 오전 9시, 전 직원이 동시에 사내 시스템 접속

증상

- 대시보드 로딩: 평소 2초 → **월요일 15초**
- DB 커넥션 풀 고갈로 일부 사용자 **접속 불가**
- 화요일부터는 정상 복구

원인: 특정 시간대 집중 접속 패턴 미반영

> **교훈**: "평균 사용자 수"가 아닌
> **"피크 시간대의 동시 사용자 수"** 를 기준으로 설계해야 합니다.

© 2026 Performance Testing Workshop

사례에서 배우는 공통 교훈

1. 실제 사용 패턴을 반영한 시나리오를 **설계**하라
2. 예상 트래픽의 **2~3배** 수준으로 테스트하라
3. 전체 사용자 흐름 **(End-to-End)** 을 테스트 범위에 포함하라
4. 피크 시간대, 이벤트 등 **특수 상황**을 반드시 고려하라
5. 결과를 **수치로 기록**하고, 기준(Baseline)을 만들어라

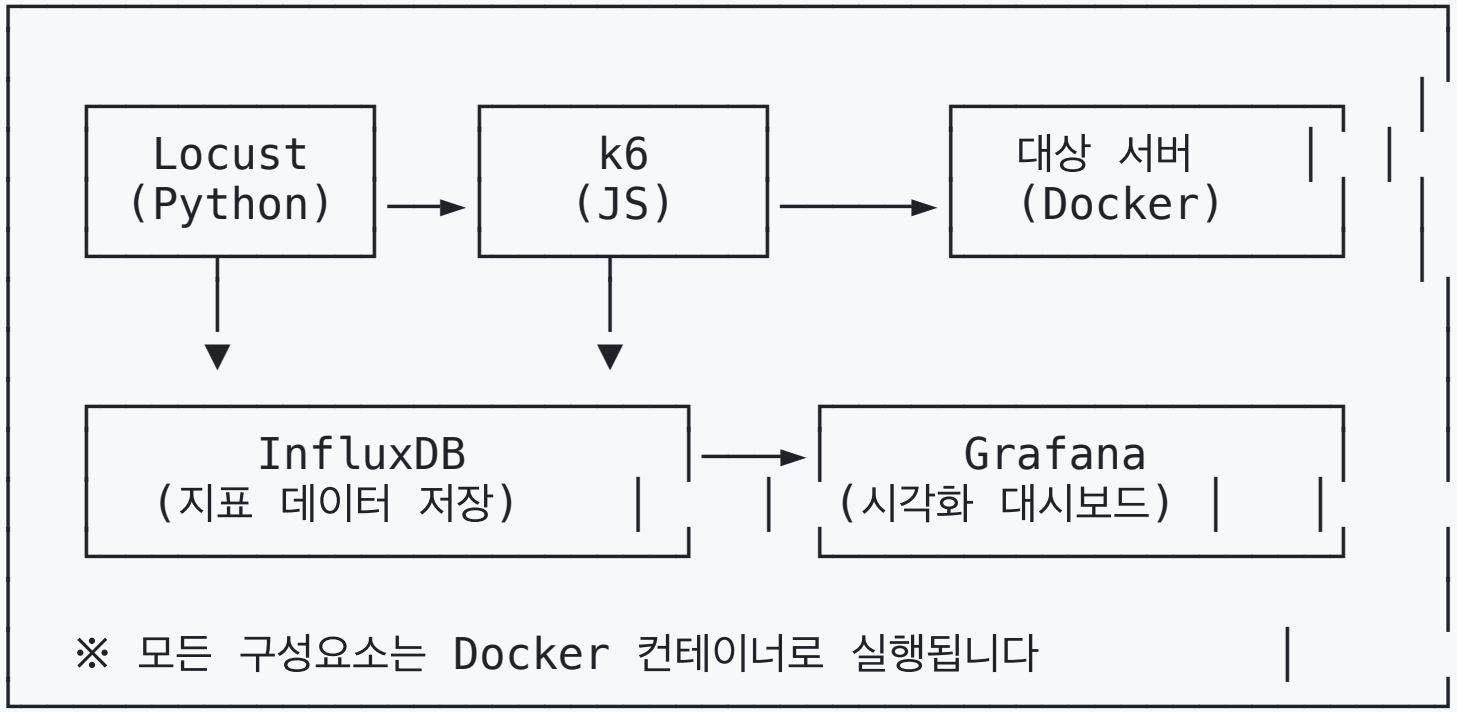
© 2026 Performance Testing Workshop

4. 실습 전에 확인하기

환경 점검

© 2026 Performance Testing Workshop

실습 아키텍처



© 2026 Performance Testing Workshop

필수 설치 도구

① Docker Desktop

- Locust, k6, InfluxDB, Grafana를 **컨테이너**로 통합 실행
- Windows: **'Use WSL 2 instead of Hyper-V'** 옵션 반드시 체크

② Visual Studio Code

- 성능 테스트 시나리오(Python, JS) 작성 에디터

설치 확인

```
docker --version          # Docker version 24.x.x
docker compose version    # Docker Compose version v2.x.x
```

© 2026 Performance Testing Workshop

VS Code 플러그인 & 환경 점검

추천 플러그인

플러그인	용도
Python	Locust 스크립트(`locustfile.py`) 인텔리센스
k6	k6 스크립트(`script.js`) 문법 하이라이팅

체크리스트

- [ ] Docker Desktop 설치 완료 & Running 상태
- [ ] `docker --version` 정상 출력
- [ ] `docker compose version` 정상 출력
- [ ] VS Code 설치 완료
- [ ] Python / k6 플러그인 설치

© 2026 Performance Testing Workshop

5. 성능 테스트 해보기

Locust

© 2026 Performance Testing Workshop

Locust란?

Python 기반의 오픈소스 성능 테스트 도구

항목	내용
언어	Python
특징	코드 기반 시나리오 작성, Web UI 제공
장점	Python을 아는 사람이면 빠르게 시작 가능

> 빠른 프로토타이핑과 **실시간 Web UI 모니터링**에 강점

© 2026 Performance Testing Workshop

Locust 핵심 구조

```python
from locust import HttpUser, task, between

class MyUser(HttpUser):
    # Think Time: 1~3초 사이 랜덤 대기
    wait_time = between(1, 3)

    @task
    def my_task(self):
        self.client.get("/api/endpoint")
```

구성 요소	설명
`HttpUser`	가상 사용자(VU) 정의 클래스
`wait_time`	Think Time 설정
`@task`	VU가 반복 수행할 작업
`self.client`	HTTP 요청 클라이언트

© 2026 Performance Testing Workshop

실습: 기본 부하 테스트 — 스크립트

```python
from locust import HttpUser, task, between

class BasicUser(HttpUser):
    """기본 부하 테스트 시나리오"""
    wait_time = between(1, 3)  # 요청 간 대기 시간

    @task(3)  # 가중치 3: 더 자주 실행됨
    def get_homepage(self):
        """메인 페이지 조회"""
        self.client.get("/", name="메인페이지")

    @task(1)  # 가중치 1: 상대적으로 적게 실행됨
    def get_api(self):
        """API 엔드포인트 조회"""
        self.client.get("/api/data", name="API조회")
```

> **@task 가중치**: `@task(3)` vs `@task(1)` → 메인페이지가 **3배 더 자주** 실행

© 2026 Performance Testing Workshop

실습: 기본 부하 테스트 — 실행

Step 1: Docker로 실행

```
docker compose up locust
```

Step 2: Web UI 접속 (`http://localhost:8089`)

항목	설명	예시 값
Number of users	최대 VU 수	10
Ramp up	초당 추가 사용자 수	2
Host	대상 서버 주소	http://target-server:8080

Step 3: Start 버튼 클릭!

© 2026 Performance Testing Workshop

실습: 시나리오 기반 테스트

```
from locust import HttpUser, task, between, SequentialTaskSet

class UserBehavior(SequentialTaskSet):
    """로그인 → 목록 조회 → 상세 조회 (순서대로 실행)"""

    def on_start(self):
        """시나리오 시작 시 1회 실행: 로그인"""
        self.client.post("/api/login", json={
            "username": "testuser", "password": "testpass"
        }, name="1.로그인")

    @task
    def view_list(self):
        self.client.get("/api/products", name="2.목록조회")

    @task
    def view_detail(self):
        self.client.get("/api/products/1", name="3.상세조회")

class ScenarioUser(HttpUser):
    wait_time = between(2, 5)
    tasks = [UserBehavior]
```

© 2026 Performance Testing Workshop

Locust 결과 읽는 법

Web UI 주요 탭

탭	확인 내용
Statistics	요청 수, 평균/p95/p99 응답 시간, 에러율
Charts	응답 시간·RPS·사용자 수 변화 그래프
Failures	실패 요청의 에러 메시지, 상태 코드

결과 해석 예시

지표	값	판단
전체 에러율	0.3%	1% 이하 → **안정적**
메인페이지 p95	250ms	95% 사용자 250ms 이내 → **양호**
API 조회 p99	1200ms	1% 사용자 1.2초 대기 → **검토 필요**

© 2026 Performance Testing Workshop

6. 성능 테스트 잘 해보기

k6

© 2026 Performance Testing Workshop

k6란?

JavaScript(ES6) **기반**의 현대적인 성능 테스트 도구

항목	내용
언어	JavaScript (ES6)
특징	CLI 기반, 코드로 모든 것을 정의
장점	세밀한 시나리오 제어, CI/CD 연동, **자동 Pass/Fail 판정**

> 자동화 파이프라인과 **정밀한 성능 기준 관리**에 강점

© 2026 Performance Testing Workshop

Locust vs k6

비교 항목	Locust	k6
언어	Python	JavaScript
실행 방식	Web UI + CLI	CLI 중심
시각화	Web UI 내장	Grafana 연동
자동 판정	수동 확인	**Thresholds로 자동 Pass/Fail**
CI/CD 연동	가능	**매우 용이**
적합한 상황	빠른 프로토타이핑	정밀한 시나리오, 자동화

© 2026 Performance Testing Workshop

k6 핵심 구조

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 10,            // 가상 사용자 10명
  duration: '30s',    // 30초간 테스트
};

export default function () {
  const res = http.get('http://target-server:8080/');

  check(res, {
    '상태코드 200': (r) => r.status === 200,
    '응답시간 500ms 이하': (r) => r.timings.duration < 500,
  });

  sleep(1);  // Think Time
}
```

© 2026 Performance Testing Workshop

실습: Stages를 활용한 부하 테스트

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '1m', target: 10 },  // Ramp-up: 0→10명 (1분)
    { duration: '3m', target: 10 },  // Steady: 10명 유지 (3분)
    { duration: '1m', target: 0 },   // Ramp-down: 10→0명 (1분)
  ],
  thresholds: {
    http_req_duration: ['p(95)<500', 'p(99)<1000'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const res = http.get('http://target-server:8080/');
  check(res, { '상태코드 200': (r) => r.status === 200 });
  sleep(Math.random() * 2 + 1);  // 1~3초 랜덤 대기
}
```

© 2026 Performance Testing Workshop

Thresholds — 자동 Pass/Fail 판정

k6의 가장 강력한 기능!

```
thresholds: {
  http_req_duration: ['p(95)<500'],          // p95 응답 시간 500ms 이하
  http_req_failed: ['rate<0.01'],            // 에러율 1% 미만
  'http_req_duration{name:로그인}': ['p(95)<300'],  // 로그인 API 별도 기준
},
```

실행 결과

```
✓ http_req_duration.....: avg=120ms  p(95)=350ms  p(99)=800ms
  ✓ { p(95)<500 } ──────── pass ✓
✓ http_req_failed.......: 0.25%
  ✓ { rate<0.01 } ────── pass ✓
```

> 모두 통과 → 종료 코드 `0` (성공) / 하나라도 실패 → 종료 코드 `99`
>
> CI/CD에서 **자동 합격/불합격 판정** 가능!

© 2026 Performance Testing Workshop

실습: 시나리오 기반 테스트

```
import http from 'k6/http';
import { check, group, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 5 }, { duration: '2m', target: 5 },
    { duration: '30s', target: 0 },
  ],
};

export default function () {
  group('01_로그인', function () {
    const res = http.post('http://target-server:8080/api/login',
      JSON.stringify({ username: 'testuser', password: 'testpass' }),
      { headers: { 'Content-Type': 'application/json' } });
    check(res, { '로그인 성공': (r) => r.status === 200 });
    sleep(1);
  });

  group('02_목록조회', function () {
    const res = http.get('http://target-server:8080/api/products');
    check(res, { '목록 조회 성공': (r) => r.status === 200 });
    sleep(2);
  });
}
```

© 2026 Performance Testing Workshop

Grafana로 결과 시각화

k6 → InfluxDB → Grafana 연동

```
k6 run --out influxdb=http://localhost:8086/k6 script.js
```

Grafana 대시보드에서 확인 가능한 항목

- 실시간 **응답 시간 추이** (평균, p95, p99)
- **초당 요청 수 (RPS)** 변화 그래프
- **에러율** 변화 추이
- **가상 사용자 수** 변화

> 숫자만으로는 파악하기 어려운 **추세와 패턴**을 시각적으로 확인!

© 2026 Performance Testing Workshop

k6 결과 읽는 법

```
          /\      |‾‾| /‾‾/   /‾‾/
     /\  /  \     |  |/  /   /  /
    /  \/    \    |     (   /   ‾\
   /          \   |  |\  \ |  (‾)  |
  / __________ \  |__| \__\ \_____/

  checks.......................: 100.00% ✓ 450  ✗ 0
  http_req_duration...........: avg=135ms  p(90)=250ms  p(95)=380ms
  http_req_failed.............: 0.00%  ✓ 0    ✗ 600
  http_reqs...................: 600    3.33/s
  iterations..................: 150    0.83/s
```

지표	값	해석
`checks`	100% ✓	모든 응답 검증 **통과**
`http_req_duration p(95)`	380ms	95% 요청이 380ms 이내 → **양호**
`http_req_failed`	0.00%	에러 없음 → **안정적**
`http_reqs`	3.33/s	초당 약 3.3건 처리 (RPS)

© 2026 Performance Testing Workshop

어떤 도구를 선택할까?

Locust를 선택하세요	k6를 선택하세요
Python이 익숙할 때	JavaScript가 익숙할 때
Web UI로 빠르게 테스트할 때	CI/CD 파이프라인 연동 시
프로토타이핑/탐색적 테스트 시	자동 Pass/Fail 판정 필요 시
팀 내 Python 사용자가 많을 때	정밀한 시나리오 제어 필요 시

> 두 도구 모두 실무에서 널리 사용됩니다.
> **정답은 없으며**, 팀의 기술 스택과 상황에 맞게 선택하세요.

© 2026 Performance Testing Workshop

수고하셨습니다!

서버의 한계를 측정하고, 장애를 예방하는 첫 걸음을 내딛었습니다.

질문이 있으시면 편하게 말씀해 주세요.

© 2026 Performance Testing Workshop